MCP 통합 Agent (실습)

5-1. Agent에 MCP 서버 연결 (실습)
5-2. 통합 시나리오 검증 (실습)

5-1. Agent에 MCP 서버 연결 (실습)

> 검증된 MCP 서버를 LangGraph Agent에 붙여 일반 도구와 섞어 써봅니다.

이제 검증이 끝난 MCP 서버를 LangGraph Agent에 붙입니다. 다리 역할은 langchain-mcp-adapters 패키지의 MultiServerMCPClient입니다. 하는 일은 두 가지입니다.

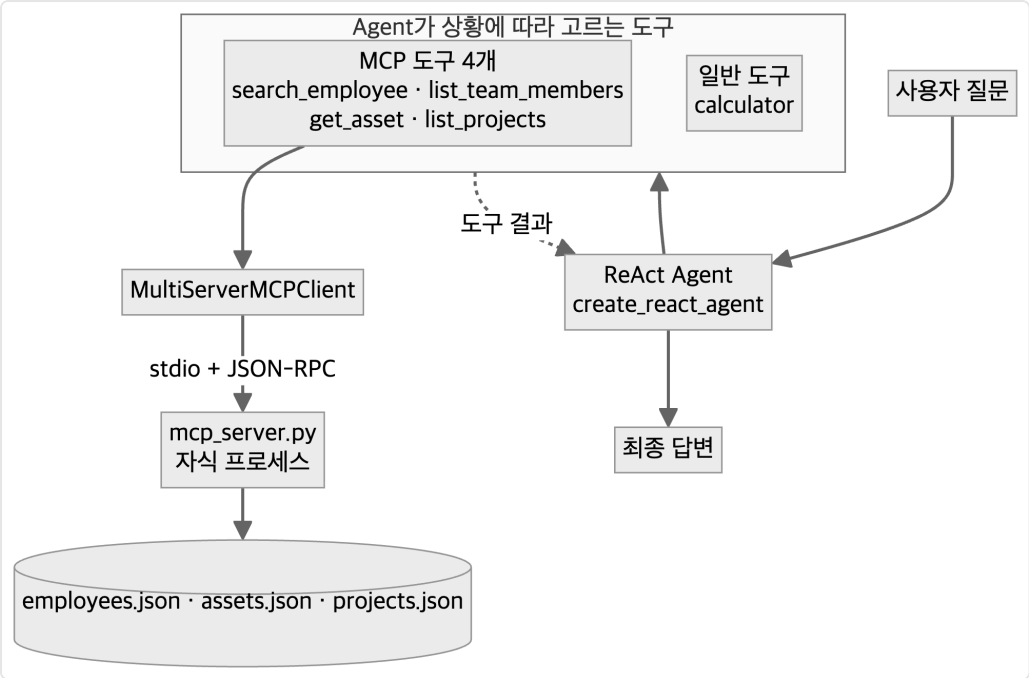
1. 설정에 적힌 MCP 서버들을 연결 (stdio라면 자식 프로세스로 실행)
2. 각 서버의 MCP 도구들을 **LangChain Tool 객체로 변환**해서 돌려주기

변환이 끝나면 MCP 도구는 첫 번째 세션에서 만든 @tool 도구와 완전히 같은 방식으로 쓸 수 있습니다. 일반 @tool 도구(계산기 등)와 한 리스트에 섞어 쓰는 것도 됩니다. 이것이 어댑터의 가치입니다.

이름에서 보이듯 서버 여러 개를 동시에 연결할 수도 있습니다.

Agent 본체는 langgraph의 create_react_agent를 씁니다. 2번째 세션 2-2에서 StateGraph, ToolNode, 조건 분기로 직접 조립했던 ReAct 구조를 이 함수 한 줄이 그대로 만들어 줍니다. 원리를 이미 손으로 봤으므로 이제는 완성된 조립기를 쓰면 됩니다.

전체 흐름은 다음과 같습니다.


Agent가 상황에 따라 고르는 도구
MCP 도구 4개
search_employee · list_team_members
get_asset · list_projects
일반 도구
calculator
사용자 질문
도구 결과
MultiServerMCPClient
ReAct Agent
create_react_agent
stdio + JSON-RPC
mcp_server.py
자식 프로세스
최종 답변
employees.json · assets.json · projects.json

따라하기

1. 통합 Agent 파일을 만듭니다. 아래 코드를 `day4_practice/mcp_agent.py` 로 저장합니다. MCP 도구의 실제 실행이 비동기라서 실행도 `ainvoke` (invoke의 비동기판)를 씁니다.

```
# mcp_agent.py - MCP 서버 + 일반 도구를 함께 쓰는 LangGraph 통합 Agent
import asyncio
import os
import sys
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent

load_dotenv()


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


SERVER_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "mcp_
server.py")

SYSTEM_PROMPT = """당신은 삼성SDS 사내 어시스턴트입니다. 교육용 더미 데이터를 조회해 답합니
다.

[도구 사용 규칙]
- 임직원 개인 정보 -> search_employee
- 팀 인원수, 팀 명단 -> list_team_members
- 자산 현황 -> get_asset
- 프로젝트 현황 -> list_projects
- 수치 계산 -> calculator (암산하지 말고 반드시 도구 사용)

도구에서 에러 메시지가 오면 그 지침을 따라 수정해서 다시 시도하세요.
도구 결과에 근거해서만 답하세요."""


# MCP 도구와 섞어 쓸 일반 도구
@tool
def calculator(expression: str) -> str:
    """수학 계산을 정확하게 수행한다. 예: '3 * 2000000'"""
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
```

```
        return f"에러: 계산식이 잘못되었습니다 ({e}). 올바른 수식으로 다시 호출하세요."


async def main():
    # 1) MCP 클라이언트 설정: 서버 이름 -> 실행 방법
    client = MultiServerMCPClient({
        "sds-company-data": {
            "command": sys.executable,
            "args": [SERVER_PATH],
            "transport": "stdio",
        },
        # 서버를 더 붙이고 싶으면 여기에 항목 추가
    })

    # 2) MCP 도구를 LangChain Tool로 변환해 가져오기 (await 필수)
    mcp_tools = await client.get_tools()
    print(f"MCP 도구 {len(mcp_tools)}개: {[t.name for t in mcp_tools]}")

    # 3) 일반 도구와 합치기: 여기부터는 MCP 여부 구분이 사라집니다
    tools = mcp_tools + [calculator]

    # 4) 2-2에서 손으로 조립한 ReAct 구조를 한 줄로
    llm = ChatBedrockConverse(
        model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
        region_name="us-east-1",
    )
    agent = create_react_agent(llm, tools, prompt=SYSTEM_PROMPT)

    # 5) 실행: MCP 도구가 섞여 있으므로 비동기 호출(ainvoke) 사용
    result = await agent.ainvoke(
        {"messages": [("user", "김하늘 님이 무슨 자산을 갖고 있는지 알려주세요.")]}
    )
    print(f"\n답변: {get_text(result['messages'][-1])}")


if __name__ == "__main__":
    asyncio.run(main())
```

2. 실행하면 MCP 도구 4개가 로드되고 자산 답변이 나옵니다.

```
MCP 도구 4개: ['search_employee', 'list_team_members', 'get_asset', 'list_proj
ects']

답변: 김하늘 님은 현재 노트북(갤럭시북4 프로)과 모니터(뷰피니티 S8) 총 2건의 자산을 사용 중입니
다.
```

막힐 때

- coroutine was never awaited 경고가 나옵니다.

  `client.get_tools()` 앞에 `await`를 빼먹은 경우입니다. 도구 리스트 대신 코루틴 객체가 와서 뒤 코드가 전부 이상해집니다. MCP 관련 호출은 전부 async라는 것만 기억하면 됩니다.

- 도구 실행 단계에서 에러가 납니다.

  MCP 도구가 섞인 Agent는 `agent.invoke()` (동기) 대신 `await agent.ainvoke()`를 씁니다. MCP 도구의 실제 실행이 비동기이기 때문입니다.

- 반복 실행할 때 매번 `get_tools()`를 부르면 서버 프로세스가 계속 새로 뜹니다: 도구 목록은 프로그램 시작 시 한 번만 가져와 재사용합니다.

- `ImportError: cannot import name 'create_react_agent'`가 납니다.

  최신 langgraph에서 이름이 `create_agent`로 바뀌었습니다. `from langchain.agents import create_agent`로 바꿔 쓰되 프롬프트 인자 이름이 `prompt`가 아니라 `system_prompt` 입니다. 5일차에서 이 API를 본격적으로 씁니다.

확인: MCP 도구 4개가 로드되고 자산 질문에 get_asset 결과 기반의 답변이 나오면 성공입니다.

> 어댑터가 MCP 도구를 LangChain Tool로 바꿔 주면 일반 도구와 구분 없이 쓸 수 있다는 걸 확인했습니다.

5-2. 통합 시나리오 검증 (실습)

> 사내 데이터 조회와 일반 도구를 엮는 복합 질문으로 도구 선택과 체이닝을 검증해봅니다.

통합 Agent가 진짜 쓸모 있는지는 여러 도구를 이어야만 답할 수 있는 복합 질문으로 확인합니다. 단일 도구 질문과 복합 질문을 섞어 던지고 도구 호출 순서를 실시간으로 관찰합니다. 이번에는 `astream` (stream의 비동기판)으로 실행 도중의 노드별 업데이트를 받아 trace를 눈으로 봅니다.

여기서 `stream_mode`가 처음 나오므로 짚고 갑니다. `astream()`은 실행이 다 끝나기를 기다리지 않고 진행 상황을 이벤트로 흘려보냅니다. `stream_mode`는 **그 이벤트를 어느 단위로 받을지** 고르는 옵션입니다.

값	받는 것	어느 때 쓰나
`values`	단계마다 State 전체 스냅샷	현재 상태 전체를 통째로 보고 싶을 때
`updates`	방금 실행된 노드의 변경분만 (노드 이름 → 바뀜 내용)	어느 단계가 무엇을 했는지 추적할 때
`messages`	LLM이 생성하는 토큰	챗봇 UI의 타이핑 효과를 낼 때

이번에 `updates`를 고른 이유는 목적이 "모델이 어떤 도구를 어떤 순서로 불렀는가"를 보는 것이기 때문입니다. `values`로 받으면 단계마다 `messages` 배열 전체가 다시 오기 때문에 이미 본 메시지까지 매번 다시 훑으며 새로 생긴 것을 과거 목록과 비교해 걸러내야 합니다.
반면 `updates`는 방금 추가된 것만 오므로 그대로 출력하면 됩니다. 아래 코드가 `event.items()`로 노드 이름과 변경분을 꺼내 쓰는 것도 이 형식 덕분입니다.

따라하기

1. `mcp_agent.py`의 main 함수에서 5) 실행 부분을 다음으로 교체합니다. 붙여넣는 코드는 main 함수 안에 들어가므로 들여쓰기를 함수 본문에 맞춥니다.

```
# 5) 단일 도구 질문과 복합 질문을 섞어 검증
queries = [
    # 단일 MCP 도구
    "박도윤 님의 이메일 주소를 알려주세요.",
    # 팀 단위 조회 (list_team_members)
    "클라우드운영팀 인원이 몇 명인지 알려주세요.",
    # MCP 도구 두 개를 이어야 하는 질문
    "이서준 님이 소속된 팀에서 진행 중인 프로젝트를 알려주세요.",
    # MCP 조회 + 일반 계산 도구를 엮는 복합 질문
    "노트북 자산이 총 몇 대인지 세고 한 대당 200만원이면 총액이 얼마인지 계산해 주세요.",
]

for q in queries:
    print(f"\n{'=' * 60}\n질문: {q}\n{'=' * 60}")
    async for event in agent.astream(
        {"messages": [("user", q)]},
        stream_mode="updates",
    ):
        for node, update in event.items():
            for m in update.get("messages", []):
                if getattr(m, "tool_calls", None):
                    for tc in m.tool_calls:
                        print(f"  도구 호출: {tc['name']}({tc['args']})")
                elif m.type == "tool":
                    print(f"  도구 결과: [{m.name}] {get_text(m)[:80]}")
                elif m.content:
                    print(f"  답변: {get_text(m)[:200]}")
```

2. 실행해서 세 번째 질문의 trace를 관찰합니다. 조회 결과가 다음 도구의 인자로 흘러가는 것이 핵심입니다.

```
============================================================
질문: 노트북 자산이 총 몇 대인지 세고 한 대당 200만원이면 총액이 얼마인지 계산해 주세요.
============================================================
  도구 호출: get_asset({'owner': ''})
  도구 결과: [get_asset] {"count": 8, "assets": [{"asset_id": "A-1001", ...
  도구 호출: calculator({'expression': '5 * 2000000'})
  도구 결과: [calculator] 10000000
  답변: 노트북 자산은 총 5대이며 한 대당 200만원으로 계산하면 총액은 1,000만원입니다.
```

3. trace에서 세 가지를 점검합니다.
   - 질문 의도에 맞는 도구가 선택되었는가 (엉뚱한 도구가 불리면 1번째 세션 1-1의 설명 개선으로 돌아갑니다)
   - 도구가 올바른 순서로 이어졌는가 (조회 결과가 다음 도구의 인자로 활용되는가)
   - 최종 답변이 도구 결과에 근거하는가 (도구 결과에 없는 숫자를 지어내면 시스템 프롬프트의 "도구 결과에 근거해서만 답하세요"를 강화합니다)

막힐 때

- 복합 질문에서 모델이 계산을 도구 없이 암산으로 해 버립니다.

  시스템 프롬프트에 "수치 계산은 반드시 calculator를 사용"이 있는지 확인합니다. 그래도 새면 문구를 더 단호하게 바꿉니다.

- 도구 결과가 길어서 모델이 일부만 보고 잘못 셉니다: 도구 쪽에서 요약된 형태로 반환하는 것이 근본 대책입니다. get_asset이 count를 함께 반환하는 이유가 이것입니다.

확인: 단일 도구 질문은 도구 1개, 팀 프로젝트 질문은 search_employee와 list_projects 2개, 총액 질문은 get_asset과 calculator가 순서대로 이어지면 성공입니다.

> 같은 Agent가 질문에 따라 다른 도구를 고르고 조회 결과를 다음 도구로 잇는 것을 trace로 확인했습니다.